

# Evaluating a Deterministic Validator Plus Single-Iteration Repair Pipeline for LLM-Generated Network Configurations

Autonomous AI Researcher  
CNSM 2026 Agentic AI Researcher Track

**Abstract**—We preregistered and executed a paired confirmatory experiment (N=300 paired intent→configuration tasks; pre-registration id cand-2026-001-A-prereg-v1, preregistration SHA-256 archived) to evaluate whether a minimal guarded pipeline—single-shot LLM generation (declared model: gpt-5-mini) followed by a frozen deterministic configuration validator and at most one automated repair iteration—reduces first-pass misconfiguration relative to plain single-shot generation. Execution completed 600 episodes and used 301 model calls. Observed outcomes: baseline = 299/300 valid (99.667%); guarded = 300/300 valid (100.0%); paired counts n11=299, n10=1, n01=0, n00=0. Exact McNemar two-sided  $p = 1.00$ ; absolute paired difference = 0.0033333; paired bootstrap 95% CI = [0.00, 0.01] (10,000 resamples, seed 1729). The single permitted automated repair corrected the sole invalid baseline output (repair success 1/1; Clopper–Pearson 95% CI = [0.025, 1.0]). The preregistered planning assumption used for power calculations (planned baseline pass rate = 0.40) was contradicted by the observed near-ceiling baseline, removing statistical headroom for the preregistered  $\geq 50\%$  relative reduction target. We report preregistered design choices, execution accounting, confirmatory statistics, a post-hoc inferential sensitivity bound tied to the observed contingency counts, and artifact-grounded recommendations for future NetOps confirmatory evaluations. All run artifacts and analysis outputs are archived in the execution manifest referenced in the Disclosure Statement.

## I. INTRODUCTION

Translating operator intent to device configuration is a core NetOps task. Large language models (LLMs) have been proposed as intent→configuration translators that can accelerate operations, but probabilistic generation risks misordering, omission, and unsafe commands. Recent literature advocates hybrid patterns that pair probabilistic generators with deterministic validators/simulators and bounded repair loops as operational floor-safety nets. However, rigorous preregistered evaluations that quantify how tightly bounded guarded pipelines change first-pass misconfiguration rates under controlled sampling semantics are limited.

We implement and preregister a narrowly scoped paired experiment to evaluate the operational effect of a minimal guarded pipeline composed of: (1) a single shared LLM candidate per task (single-shot generation), (2) a frozen deterministic validator performing canonicalization and rule/dependency checks, and (3) at most one automated repair attempt when the validator reports violations. The core research question is: Can this minimal guarded pipeline reduce first-pass misconfiguration rate relative to plain single-shot generation under a

controlled paired design? Our contributions are (i) a preregistered paired experiment with deterministic seed generation and archived per-task artifacts, (ii) confirmatory statistics derived from those artifacts, (iii) a compact post-hoc sensitivity quantification tied to the observed contingency counts, and (iv) artifact-grounded recommendations for future NetOps confirmatory evaluations.

## II. RELATED WORK

Work applying LLMs to network configuration and intent translation has grown rapidly. Recent system demonstrations show promising intent→configuration synthesis in lab settings but also document failure modes—misordering, omission, and overconfidence—that affect operational safety [10.1002/nem.2313]. Parallel work in adjacent domains (medical QA, code repair) shows that verifier-assisted and retrieval-augmented pipelines reduce factual errors and improve grounding, motivating similar guarded architectures for NetOps [10.36227/techrxiv.176784505.56675782/v1, 10.2139/ssrn.6608518]. Classic deterministic network verification (header-space analysis, specification-based checking) provides algorithmic foundations for validators that can statically check reachability and policy constraints without live execution [10.1145/3098822.3098834]. Survey and review articles emphasize the generate→verify→repair design pattern and highlight auditability and human-oversight concerns as open challenges for production NetOps adopters [10.6028/nist.ai.100-2e2023]. Our study positions itself at the intersection of these strands by executing a preregistered paired evaluation that directly measures first-pass validity under an explicit validator+repair floor-guardrail.

## III. METHODOLOGY

Preregistration and confirmatory hypotheses: The experiment was preregistered under id cand-2026-001-A-prereg-v1 before any final outcomes were observed. Two confirmatory hypotheses were registered: H1 — the guarded pipeline reduces first-pass misconfiguration rate by  $\geq 50\%$  (relative) versus single-shot generation; H2 — one or fewer automated repair iterations recover  $\geq 75\%$  of initially invalid configurations. The preregistration explicitly documented the numeric planning assumption used for power calculations: planned baseline pass rate = 0.40. This planning assumption informed

the targeted sample size and the preregistered effect magnitude and is reported here for transparent interpretation.

**Design and sampling semantics:** We used a paired within-task design with a fixed deterministic seed list of 300 tasks produced by a frozen task generator. Each seed produced an intent and an associated canonical reference configuration; seeds were included deterministically unless a preregistered parser exclusion rule applied (such replacements are recorded in the archived manifest). For each seed the same single LLM candidate was used in both arms (shared\_initial\_candidate semantics): the generator produced one configuration (shared candidate), which served as the baseline response and was the starting point for the guarded arm's validation and possible repair. Planned episodes = 600 (two episodes per seed), initial\_generation\_calls\_per\_task = 1, maximum\_repair\_calls\_per\_task = 1. The shared-candidate pairing was preregistered to isolate the incremental effect of deterministic validation and bounded repair on a single proposed plan; as noted below, this choice increases within-pair correlation and reduces the number of statistically informative discordant pairs available to paired tests compared with an independent-draws design.

**Model, resource constraints, and provider-call policy:** The declared generation model in the execution contract was gpt-5-mini. The execution contract limited total model calls to a planned maximum of 600 (one initial generation per seed across 300 seeds plus up to one repair per seed). Provider call failures are handled per the preregistered retry policy (single retry allowed for provider/infrastructure failures); the primary missingness treatment drops pairs only if both calls in a pair failed ('complete\_pair\_primary'). The archived model configuration artifact execution/model\_configuration.json (sha256: 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923fa) records the declared\_model\_version as "gpt-5-mini" and records "temperature": null. We explicitly note that a null or unset temperature field in the archived configuration is not evidence of deterministic sampling (temperature=0); sampling determinism was therefore unspecified by the archived runtime configuration and generator variance could be affected by nonfixed sampling settings. This unspecified sampling determinism is a documented inferential caveat because pairing and expected discordant counts depend in part on generator variance.

**Validator canonicalization, rules, and scoring semantics:** The frozen deterministic validator implemented canonicalization to a normalized configuration AST and applied a suite of static checks derived from explicit workflow and dependency policies encoded in the task payloads. The scoring rule used across the experiment was full intent-constraint validation: a generated configuration is scored as success only when (a) the validator's normalized\_configuration satisfies the task's required\_sequence and expected\_final\_state constraints, and (b) all preregistered workflow policies (e.g., 'must\_be\_down\_for\_fields', 'final\_group\_constraints',

and 'minimum\_active\_in\_groups') are satisfied. The validator's trace records parsed\_command\_count, required\_command\_count, observed\_touched\_field\_count, violation\_codes, and a boolean valid flag; these elements were the operational primitives used to trigger repair and to compute per-task scores. Canonicalization enables deterministic AST equality and normalized-string comparisons used in contamination detection and exact matching checks.

**Repair adapter behavior and steering:** The preregistered automated repair adapter is invoked only when the validator reports violations on the shared candidate. The repair adapter receives the shared candidate plus structured violation codes and is restricted to a single automated repair attempt per episode. Its objective is minimal, targeted editing of the shared candidate sufficient to remove the reported violations while preserving as much of the original plan as permitted by task constraints. Repair prompts and repair responses were archived per episode; the repair outcome was revalidated by the same deterministic validator to produce the final guarded decision. Because the repair budget was intentionally bounded to one call, repair-effectiveness estimands are conditional on the initially invalid subset and must be interpreted with that budget constraint in mind.

**Contamination checks, exclusions, and replacement rules:** The preregistration included deterministic contamination detection based on exact normalized-AST equality and deterministic near-duplicate lexical thresholds; any exact duplicates between task targets and transformation/unit-test artifacts would trigger deterministic exclusion and deterministic replacement from a preregistered reserve set. The primary analysis uses the decontaminated paired set produced by these rules; sensitivity analyses are specified to report results on the full set when flags arise. All exclusions, replacements, and contamination flags are recorded in the archived manifest per the preregistered plan.

**Primary estimand and statistical analysis plan:** The primary estimand was the paired\_success\_rate\_difference\_guarded\_minus\_baseline (absolute difference in per-task success probability under guarded vs baseline). The preregistered primary test was the exact McNemar two-sided test at  $\alpha=0.05$  on paired binary outcomes; paired bootstrap percentile 95% CIs for the absolute paired difference were preregistered (10,000 resamples, explicit seed). Secondary analyses included exact binomial CIs for repair success conditional on initial failure, stratified paired checks by task difficulty and constrained vendor templates, and sensitivity treatments for failed provider calls (treat as failures or exclude per the preregistered missingness plan). The preregistration contained a power/precision plan that translated the planned baseline pass rate = 0.40 and a target relative reduction ( $\geq 50\%$  relative reduction in failure rate) into a required N under conservative discordant assumptions; that planning arithmetic is reported in the preregistration record and is discussed below because the observed baseline diverged from that assumption.

Reproducibility and archival primitives: The experiment preregistered and archived all deterministically generated seeds, the task generator configuration, the scoring rules, validator trace formats, repair prompt templates, the analysis code, and the execution manifest. Key reproducibility primitives include the normalized AST canonicalization used by the validator (the basis for deterministic equality checks), the pair sampling semantics (shared candidate per pair), the maximum repair budget (one per episode), the missingness/retry policy, and the primary exact McNemar test with bootstrap CI parameters (10,000 resamples, explicit seed). These artifacts are enumerated in the execution manifest and analysis bundle referenced in the Disclosure Statement and constitute the authoritative sources for the numeric counts and inferential inputs reported in this paper.

#### IV. RESULTS

Execution accounting (artifact-derived): Planned episodes = 600; completed episodes = 600; complete pairs analyzed = 300. Model calls used = 301 (300 shared initial generations + 1 repair invocation). No provider or infrastructure failures occurred under the preregistered retry policy; no deterministic exclusions for contamination were required under the preregistered checks. Per-task normalized responses, validator decisions, and repair traces are archived and are the direct source of the numerical counts below.

Primary outcomes (archived counts): Baseline\_success\_count = 299/300 (99.667%); Guarded\_success\_count = 300/300 (100.0%). Paired contingency (guarded vs baseline) derived from archived analysis artifacts:  $n_{11} = 299$  (both success),  $n_{10} = 1$  (guarded success, baseline fail),  $n_{01} = 0$  (guarded fail, baseline success),  $n_{00} = 0$  (both fail). These counts are reproduced in the archived paired contingency artifact used for confirmatory inference.

Inferential results (preregistered): Exact McNemar two-sided test on the observed contingency gives  $p = 1.00$ . The observed absolute paired difference (guarded - baseline success rate) =  $1/300 \approx 0.003333$ . The paired bootstrap percentile 95% CI for the absolute paired difference =  $[0.00, 0.01]$  (10,000 resamples, bootstrap seed = 1729). Repair outcomes (H2): the single allowed automated repair corrected the sole invalid baseline output (repair success 1/1); Clopper-Pearson 95% CI for 1/1 is  $[0.025, 1.0]$ , indicating wide uncertainty given the single observed repair.

Post-hoc sensitivity bound (artifact-grounded): The total number of discordant pairs observed is  $n_{10} + n_{01} = 1$ . Under the paired sampling semantics used, this discordant total imposes a hard limit on the resolvable absolute paired difference equal to  $1/300 \approx 0.00333$ . Therefore (a) the maximal absolute improvement consistent with the observed discordant count is  $1/300$ ; and (b) the preregistered target corresponding to a  $\geq 50\%$  relative reduction from a planned baseline of 0.40 ( $\approx$  absolute +0.30) would have required on the order of 90 net discordant pairs favoring guarded outputs. This artifact-grounded bound explains why the preregistered large relative effect was

not detectable: the empirical baseline prevalence (299/300) left essentially no statistical headroom for the planned effect size. The archived contingency artifact and bootstrap results support these numerical statements.

Representative diagnostic artifacts: For each episode the archived validator trace contains normalized\_configuration, parsed\_command\_count, required\_command\_count, violation\_codes, and repair\_applied flags. The single baseline failure was traceable in those artifacts and was corrected by invoking the preregistered repair adapter on the shared candidate; the post-repair canonicalized configuration satisfied the full intent constraints according to the deterministic validator. These representative artifacts are available in the archived execution bundle described in the Disclosure Statement.

#### V. DISCUSSION

Quantitative interpretation tethered to archived counts. The archived contingency ( $n_{11}=299$ ,  $n_{10}=1$ ,  $n_{01}=0$ ,  $n_{00}=0$ ) directly constrains inferential outcomes: the paired difference equals  $(n_{10} - n_{01})/N = 1/300 \approx 0.00333$ , and any two-sided exact McNemar test necessarily conditions on the observed discordant count (here 1). With only one discordant pair, the sampling distribution under the null contains insufficient extreme outcomes to produce a small p-value; operationally this is why the exact McNemar test returns  $p = 1.00$  for our observed pattern. The paired bootstrap CI  $[0.00, 0.01]$  (10,000 resamples, seed 1729) expresses the same limitation in interval form: the data are consistent only with extremely small absolute improvements given the observed discordance. These numerical facts are archived and reproduce directly from the execution manifest and analysis artifacts.

Why preregistered planning assumptions matter. Our preregistered power plan assumed a baseline pass rate = 0.40; that planning choice drove an expectation of many discordant pairs under a substantial relative reduction target. The actual baseline (0.9967) collapses that expectation. The practical lesson is that power planning for generate-verify repair experiments must explicitly consider plausible baseline prevalences and the chosen pairing semantics (shared candidate vs independent draws), because both jointly determine expected discordant counts. A simple arithmetic relation is useful when translating target absolute paired differences into discordant-pair targets: absolute paired difference =  $(n_{10} - n_{01})/N$ , so for  $N=300$  an absolute increase of 0.30 requires  $(n_{10} - n_{01}) \approx 90$  net discordant pairs favoring guarded outputs. The archived artifacts provide the observed (near-zero) discordant count and therefore an artifact-grounded explanation of why the preregistered  $\geq 50\%$  relative reduction could not be detected.

Pairing semantics: operational rationale and inferential tradeoffs. We preregistered a shared\_initial\_candidate design to emulate a realistic operational workflow: operators often review and repair a single candidate plan rather than comparing independently drawn proposals. This choice improves ecological validity for the intended operational claim (effect

of validator+repair on a single generated plan) but reduces statistical signal because within-pair correlation lowers expected discordant counts. In contrast, an independent-draws design yields larger expected discordance for a given marginal success probability and thus greater power for McNemar-style paired tests; however, it addresses a different operational question (expected net benefit of switching from one generator draw to another) and increases heterogeneity. Future preregistrations should explicitly link the pairing semantics to the estimand of operational interest and incorporate that link into power and sample-size computations.

Repair effectiveness as a conditional estimand and sample-size implications. H2 framed repair success conditional on initial failure. The run observed repair success 1/1, but that conditional estimate is extremely imprecise (Clopper–Pearson 95% CI = [0.025, 1.0]). Practically, when repair effectiveness is a primary operational claim, study designers should power the experiment for the conditional probability  $P(\text{repair succeeds} \mid \text{initial failure})$  by ensuring a planned number of initial failures (not merely overall  $N$ ). This can be achieved either by calibrating task difficulty to increase baseline failure prevalence or by targeted adversarial augmentation of tasks so that the expected number of initial failures meets the sample-size requirements for the desired CI width.

Diagnostics enabled by deterministic validator traces. The frozen deterministic validator produced structured per-task traces (`normalized_configuration`, `parsed_command_count`, `required_command_count`, `violation_codes`, `repair_applied`). These traces are essential for post-hoc failure-mode analysis: they identify whether failures arise from syntax/canonicalization mismatches, missing commands, dependency-order violations, or preserved-state violations (e.g., touching protected interfaces). In our run the single baseline failure was diagnosed as a required-sequence omission by the validator and repaired by applying the preregistered repair adapter; the repair trace shows the minimal edit needed to satisfy intent constraints. Maintaining such per-task structured traces is a practical requirement for explainable NetOps guardrails and for reproducible post-mortems.

Construct validity and the role of dynamic emulation. Our validator performs deterministic static checks and canonicalization rather than executing configurations in devices or emulators; this design increases safety by avoiding live changes but limits construct validity with respect to runtime semantic properties (control-plane convergence, routing dynamics, packet-level reachability under transient state). If operational claims require guarantees about runtime consequences, future studies should integrate emulation or simulation stages that exercise control-plane and dataplane effects and record dynamic assertions (e.g., convergence time, transient reachability). The archived manifest documents that our results are strictly conditional on the validator’s static semantics.

Design recommendations grounded in observed artifacts. Based on the run artifacts and contingency counts, we recommend three concrete design levers for future confirma-

tory NetOps evaluations: (1) calibrate task difficulty or perform adversarial augmentation to target a nontrivial baseline failure prevalence or a predetermined discordant-pair count aligned with the planned absolute/relative effect size; (2) explicitly preregister pairing semantics and repair budgets and incorporate their expected effects on discordant counts into power calculations; (3) when runtime semantics matter, add an emulation/simulation validation stage and preregister how emulator outcomes map to success/failure. All of these recommendations are traceable to artifacts in the execution manifest (task difficulty fields, validator violation codes, and the paired contingency table) and are therefore actionable.

Operational implications for deploying guarded pipelines. The run shows that a frozen deterministic validator plus a bounded repair loop can correct at least some invalid candidates without human intervention (the single observed correction). However, whether such a pipeline materially reduces first-pass misconfiguration in production depends on the baseline error prevalence and the validator’s coverage of real failure modes. When baseline error is already rare, the primary operational value of a validator+repair may be as a safety net that prevents the rare catastrophic mistake rather than as a mechanism to substantially increase throughput. Organizations should therefore align evaluation goals—error-reduction vs catastrophic-risk mitigation—with study design choices (task bank selection, pairing semantics, emulation inclusion) and with the operational acceptance thresholds that govern whether automated repair may be applied without human approval.

Reproducibility and archival practice. This run demonstrates the utility of a full artifact archive: per-task inputs, normalized responses, validator traces, repair prompts/responses, the paired contingency table, and the analysis code together enable independent recomputation of reported counts and intervals. The execution manifest documents `model_calls_used` = 301 and `repair_stage` invocations = 1; these accounting numbers are minimal but sufficient to audit the reported outcomes. Future confirmatory NetOps work should adopt similar archival granularity to permit external verification and subsequent meta-analysis.

## VI. LIMITATIONS

- Synthetic task bank and constrained vendor-template scope limit external validity to broad production environments.
- Single LLM (gpt-5-mini) and a single validator implementation restrict generality across model families and validator designs.
- The preregistered power plan assumed a baseline pass rate = 0.40; the observed baseline (299/300) contradicts that assumption and removed headroom to detect the preregistered  $\geq 50\%$  relative reduction.
- Sampling semantics (`shared_initial_candidate`) reused the same initial candidate across paired arms, increasing within-pair correlation and reducing discordant pairs available to McNemar inference.

- Validation used deterministic configuration/constraint checks rather than full dynamic emulation of runtime semantic effects; actual runtime consequences of configurations were not executed and remain unmeasured.
- H2 repair-success evidence is based on a single initially invalid case (1/1  $\rightarrow$  100%), yielding a wide Clopper–Pearson 95% CI = [0.025, 1.0]; larger counts of initial failures are required to estimate repair effectiveness precisely.

## VII. CONCLUSION

We executed a preregistered paired experiment (N=300 pairs) comparing single-shot LLM generation (gpt-5-mini) to a guarded pipeline consisting of a frozen deterministic validator and a single automated repair iteration. Baseline single-shot generation yielded 299/300 valid outputs and the guarded pipeline yielded 300/300 valid outputs. The single automated repair corrected the lone invalid baseline output, but the observed near-ceiling baseline contradicted the preregistered planning assumption (baseline pass rate = 0.40) and removed statistical headroom to detect the preregistered  $\geq 50\%$  relative reduction. Future confirmatory NetOps evaluations should (a) calibrate task difficulty to produce sufficient initial failures or target discordant counts, (b) preregister pairing semantics and repair budgets explicitly, and (c) include emulation to measure runtime semantic effects when operational deployment claims are intended. All raw outputs, validator traces, repair traces, the execution manifest, and analysis artifacts are archived and enumerated in the Disclosure Statement to permit reproduction of the reported counts and intervals.

## DISCLOSURE STATEMENT

This study was executed autonomously after the autonomy lock under the archived master prompt (provenance/master\_prompt.txt; SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role: the Research Director selected the topic family (Generative AI and LLMs for NetOps) and approved the master prompt prior to the autonomy lock; no manual human editing of manuscript technical content, figures, tables, or references occurred after the lock. Preregistration: cand-2026-001-A-prereg-v1 (the preregistration record was created before observing final outcomes; preregistration SHA-256 is recorded in the execution manifest). Declared model and tooling: gpt-5-mini (declared\_model\_version), adapter family hosted\_netops\_gvr\_v1, frozen deterministic validator/repair adapter deterministic\_netops\_validator\_v5. The archived model configuration execution/model\_configuration.json (sha256: 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923fa) records the declared\_model\_version and records "temperature": null; a null/unset temperature is not evidence of deterministic sampling (temperature=0) and therefore sampling determinism was unspecified by the archived runtime configuration. This unspecified sampling determinism is a documented inferential caveat because generator variance can affect

discordant-pair expectations under paired sampling semantics. Execution accounting (archived): planned episodes = 600, completed episodes = 600, complete pairs = 300, model\_calls\_used = 301 (300 shared initial generation calls + 1 repair invocation), repair-stage invocations = 1. The execution manifest and analysis artifact bundle enumerate all raw model responses, per-task validator traces (normalized configurations, parsed\_command\_count, required\_command\_count, violation\_codes, repair\_applied flags), repair prompts/responses, execution logs, and analysis artifacts. The archived execution manifest and analysis bundle are the authoritative source for the numeric counts reported in this manuscript. The autonomous pipeline performed literature search, preregistration, execution, analysis, AI-peer-review style checks, and manuscript drafting autonomously after the autonomy lock in accordance with the CNSM Agentic AI Researcher track requirements. The preregistered planning assumption used in power calculations (planned baseline pass rate = 0.40) is explicitly reported here because it materially affects interpretation of the confirmatory result.

## REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Oghenekeno Hilkiah Okula, ""The Era of AI Agents for Network Engineering": Intent-Aware Auto Remediation for Network Disruption or Failures Using AI Agents, Large Language Models (LLM) and Model Context Protocol (MCP) Mechanisms", 2026, 10.36227/techrxiv.177004277.71795055/v1
- [3] Kunal Dhanda, "MedRAG: Retrieval-Augmented Generation for Medical QA-Comparing Base and RAG-Augmented LLM Performance on Evidence from Peer-Reviewed Clinical Research", 2026, 10.2139/ssrn.6608518
- [4] Goutam Adwant, Manjari Srivastav, "Evidence Coverage Evaluator: A Comprehensive Framework for Assessing Grounding Quality in Retrieval-Augmented Generation Systems", 2026, 10.36227/techrxiv.176784505.56675782/v1
- [5] Ryan Beckett, Aarti Gupta, Ratul Mahajan, David Walker, "A General Approach to Network Configuration Verification", 2017, 10.1145/3098822.3098834
- [6] Apostol Vassilev, Alina Oprea, Alie Jean Fordyce, Hyrum S. Anderson, "Adversarial machine learning :", 2024, 10.6028/nist.ai.100-2e2023